

Лабораторна робота №3

Тема: REST API з Express.js та інтеграційне тестування

Мета: набуття практичних навичок розробки RESTful API на Express.js з TypeScript та написання інтеграційних тестів з використанням Supertest, зокрема: • ініціалізація Express-проєкту з TypeScript та налаштування необхідних залежностей; • визначення типів сутності через Zod-схеми та z.infer; • реалізація in-memory сховища з використанням Map; • побудова CRUD-маршрутів з правильними HTTP-статус-кодами; • розділення конфігурації застосунку (app.ts) та запуску сервера (server.ts); • написання інтеграційних тестів для всіх маршрутів з використанням Supertest.

Хід роботи:

3.1. Завдання 1: Ініціалізація проєкту

```
{
  "name": "nodejs",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "dev": "ts-node src/server.ts",
    "build": "tsc",
    "test": "jest",
    "test:coverage": "jest --coverage"
  },
  "repository": {
    "type": "git",
    "url": "git+https://github.com/lhnatiukAndrii/nodejs-labs.git"
  },
  "keywords": [],
  "author": ""
}
```

					ДУ «Житомирська політехніка».26.121.8.000 – Лр3			
Змн.	Арк.	№ докум.	Підпис	Дата				
Розроб.		Ігнатюк А.М.			Звіт з лабораторної роботи	Літ.	Арк.	Аркушів
Перевір.		Фуріхата Д.В.					1	24
Керівник						ФІКТ Гр. ІПЗ-23-1		
Н. контр.								
Зав. каф.								

```

"license": "ISC",
"type": "commonjs",
"bugs": {
  "url": "https://github.com/IhnatiukAndrii/nodejs-labs/issues"
},
"homepage": "https://github.com/IhnatiukAndrii/nodejs-labs#readme",
"dependencies": {
  "cors": "^2.8.6",
  "express": "^5.2.1",
  "zod": "^4.4.3"
},
"devDependencies": {
  "@types/cors": "^2.8.19",
  "@types/express": "^5.0.6",
  "@types/jest": "^30.0.0",
  "@types/supertest": "^7.2.0",
  "jest": "^30.4.2",
  "supertest": "^7.2.2",
  "ts-jest": "^29.4.11",
  "ts-node": "^10.9.2",
  "typescript": "^6.0.3"
}
}

```

3.2. Завдання 2: Схеми та типи сутності

```

import { z } from 'zod';

export const createSchema = z.object({
  name: z.string().min(1).max(100),
  description: z.string().max(500).optional(),
  price: z.number().positive(),

```

		Ігнатюк А.М.			ДУ «Житомирська політехніка».26.121.8.000 – ЛрЗ	Арк.
		Фуріхата Д.В.				2
Змн.	Арк.	№ докум.	Підпис	Дата		

```

    priority: z.enum(['low', 'medium', 'high']),
    url: z.string().url().optional()
  });

export const updateSchema = createSchema.partial();

export type Entity = z.infer<typeof createSchema> & {
  id: string;
  createdAt: Date;
  updatedAt: Date;
};

```

3.3. Завдання 3: Модуль зберігання (in-memory storage)

```

import { Entity } from '../schemas/entity.schema';

const storage = new Map<string, Entity>();

export const entityStorage = {
  findAll(): Entity[] {
    return Array.from(storage.values());
  },

  findById(id: string): Entity | undefined {
    return storage.get(id);
  },

  create(item: Omit<Entity, 'id' | 'createdAt' | 'updatedAt>): Entity {
    const id = Math.random().toString(36).substring(2, 9);
    const now = new Date();
    const newItem: Entity = {
      ...item,
      id,

```

```

        createdAt: now,
        updatedAt: now
    };

    storage.set(id, newItem);

    return newItem;
},

update(id: string, item: Partial<Omit<Entity, 'id' | 'createdAt' | 'updatedAt'>>): Entity |
undefined {
    const existing = storage.get(id);
    if (!existing) {
        return undefined;
    }
    const updatedItem: Entity = {
        ...existing,
        ...item,
        updatedAt: new Date()
    };
    storage.set(id, updatedItem);
    return updatedItem;
},

delete(id: string): boolean {
    return storage.delete(id);
},

reset(): void {
    storage.clear();
}
};

```

3.4. Завдання 4: CRUD-маршрути та middleware

errorHandler.ts

```
import { Request, Response, NextFunction } from 'express';
import { ZodError } from 'zod';

export const errorHandler = (
  err: any,
  req: Request,
  res: Response,
  next: NextFunction
): void => {
  if (err instanceof ZodError) {
    res.status(400).json({
      status: 'error',
      message: 'Validation error',
      errors: err.issues
    });
    return;
  }

  res.status(500).json({
    status: 'error',
    message: err.message || 'Internal Server Error'
  });
};
```

validation.ts

```
import { Request, Response, NextFunction } from 'express';
import { ZodSchema } from 'zod';

export const validate = (schema: ZodSchema) => {
  return (req: Request, res: Response, next: NextFunction): void => {
```

		Ігнатюк А.М.			ДУ «Житомирська політехніка».26.121.8.000 – ЛрЗ	Арк.
		Фуріхата Д.В.				5
Змн.	Арк.	№ докум.	Підпис	Дата		

```

try {
  req.body = schema.parse(req.body);
  next();
} catch (error) {
  next(error);
}
};
};

```

index.ts

```

import { Request, Response, NextFunction } from 'express';

export const loggerMiddleware = (req: Request, res: Response, next: NextFunction) => {
  next();
};

export { validate } from './validation';
export { errorHandler } from './errorHandler';

```

src/routes/entity.ts

```

import { Router } from 'express';
import { entityStorage } from '../storage/entity';
import { createSchema, updateSchema } from '../schemas/entity.schema';
import { validate, errorHandler } from '../middleware';

const router = Router();

router.get('/', (req, res) => {
  res.status(200).json(entityStorage.findAll());
});

```

		Ігнатюк А.М.			ДУ «Житомирська політехніка».26.121.8.000 – Лр3	Арк.
		Фвріхата Д.В.				6
Змн.	Арк.	№ докум.	Підпис	Дата		

```

router.get('/:id', (req, res, next) => {
  try {
    const item = entityStorage.findById(req.params.id);
    if (!item) {
      res.status(404).json({ error: 'Item not found' });
      return;
    }
    res.status(200).json(item);
  } catch (error) {
    next(error);
  }
});

router.post('/', validate(createSchema), (req, res, next) => {
  try {
    const newItem = entityStorage.create(req.body);
    res.status(201).json(newItem);
  } catch (error) {
    next(error);
  }
});

router.put('/:id', validate(updateSchema), (req, res, next) => {
  try {
    const updated = entityStorage.update(req.params.id as string, req.body);
    if (!updated) {
      res.status(404).json({ error: 'Item not found' });
      return;
    }
    res.status(200).json(updated);
  } catch (error) {
    next(error);
  }
});

```

```

});

router.delete('/:id', (req, res, next) => {
  try {
    const success = entityStorage.delete(req.params.id);
    if (!success) {
      res.status(404).json({ error: 'Item not found' });
      return;
    }
    res.status(204).end();
  } catch (error) {
    next(error);
  }
});

router.use(errorHandler);

export default router;

```

3.5. Завдання 5: Розділення app.ts та server.ts

src/app.ts

```

import express from 'express';
import cors from 'cors';
import entityRoutes from './routes/entity';
import { loggerMiddleware, errorHandler } from './middleware';

const app = express();

app.use(cors());
app.use(express.json());
app.use(loggerMiddleware);

```

		Ігнатюк А.М.			ДУ «Житомирська політехніка».26.121.8.000 – ЛрЗ	Арк.
		Фвріхата Д.В.				8
Змн.	Арк.	№ докум.	Підпис	Дата		

```

app.use('/entities', entityRoutes);

app.use(errorHandler);

export default app;

```

src/server.ts

```

import app from './app';

const PORT = process.env.PORT || 3000;

app.listen(PORT, () => {
  console.log(`Server is running on port ${PORT}`);
});

```

3.6. Завдання 6: Інтеграційні тести з Supertest

tests/entity.test.ts

```

import request from 'supertest';
import app from '../src/app';
import { entityStorage } from '../src/storage/entity';

describe('Wishlist API Integration Tests', () => {
  beforeEach(() => {
    entityStorage.reset();
  });

  const validItem = {
    name: 'PlayStation 5',
    description: 'Next-gen gaming console',
    price: 499.99,

```

		Ігнатюк А.М.			ДУ «Житомирська політехніка».26.121.8.000 – ЛрЗ	Арк.
		Фвріхата Д.В.				9
Змн.	Арк.	№ докум.	Підпис	Дата		

```

priority: 'high' as const,
url: 'https://sony.com/ps5'
};

it('GET /entities should return an empty array initially', async () => {
  const res = await request(app).get('/entities');
  expect(res.status).toBe(200);
  expect(res.body).toEqual([]);
});

it('GET /entities/:id should return 404 for non-existent ID', async () => {
  const res = await request(app).get('/entities/non-existent-uuid');
  expect(res.status).toBe(404);
  expect(res.body).toEqual({ error: 'Item not found' });
});

it('POST /entities should create a wishlist item and return 201', async () => {
  const res = await request(app).post('/entities').send(validItem);
  expect(res.status).toBe(201);
  expect(res.body).toHaveProperty('id');
  expect(res.body).toHaveProperty('createdAt');
  expect(res.body).toHaveProperty('updatedAt');
  expect(res.body.name).toBe(validItem.name);
  expect(res.body.description).toBe(validItem.description);
  expect(res.body.price).toBe(validItem.price);
  expect(res.body.priority).toBe(validItem.priority);
  expect(res.body.url).toBe(validItem.url);
});

it('POST /entities should return 400 if name is empty', async () => {
  const res = await request(app).post('/entities').send({ ...validItem, name: '' });
  expect(res.status).toBe(400);
  expect(res.body.status).toBe('error');
});

```

```

    expect(res.body.message).toBe('Validation error');
  });

  it('POST /entities should return 400 if name exceeds 100 characters', async () => {
    const longName = 'a'.repeat(101);
    const res = await request(app).post('/entities').send({ ...validItem, name: longName });
    expect(res.status).toBe(400);
    expect(res.body.status).toBe('error');
  });

  it('POST /entities should return 400 if description exceeds 500 characters', async () => {
    const longDesc = 'a'.repeat(501);
    const res = await request(app).post('/entities').send({ ...validItem, description: longDesc });
    expect(res.status).toBe(400);
  });

  it('POST /entities should return 400 if price is negative', async () => {
    const res = await request(app).post('/entities').send({ ...validItem, price: -10 });
    expect(res.status).toBe(400);
  });

  it('POST /entities should return 400 if priority is invalid', async () => {
    const res = await request(app).post('/entities').send({ ...validItem, priority: 'critical' });
    expect(res.status).toBe(400);
  });

  it('POST /entities should return 400 if url is invalid URL format', async () => {
    const res = await request(app).post('/entities').send({ ...validItem, url: 'not-a-valid-url' });
    expect(res.status).toBe(400);
  });

  it('GET /entities/:id should return the item if it exists', async () => {
    const created = entityStorage.create(validItem);

```

```

const res = await request(app).get(`/entities/${created.id}`);
expect(res.status).toBe(200);
expect(res.body.id).toBe(created.id);
expect(res.body.name).toBe(validItem.name);
});

it('PUT /entities/:id should update wishlist item and return 200', async () => {
  const created = entityStorage.create(validItem);
  const updateData = { name: 'PlayStation 5 Slim', price: 449.99 };
  const res = await request(app).put(`/entities/${created.id}`).send(updateData);
  expect(res.status).toBe(200);
  expect(res.body.id).toBe(created.id);
  expect(res.body.name).toBe(updateData.name);
  expect(res.body.price).toBe(updateData.price);
  expect(res.body.priority).toBe(validItem.priority);
});

it('PUT /entities/:id should return 404 if updating non-existent item', async () => {
  const res = await request(app).put('/entities/non-existent-uuid').send({ name: 'New Name' });
  expect(res.status).toBe(404);
});

it('DELETE /entities/:id should delete item and return 204', async () => {
  const created = entityStorage.create(validItem);
  const delRes = await request(app).delete(`/entities/${created.id}`);
  expect(delRes.status).toBe(204);

  const getRes = await request(app).get(`/entities/${created.id}`);
  expect(getRes.status).toBe(404);
});

it('DELETE /entities/:id should return 404 if deleting non-existent item', async () => {
  const res = await request(app).delete('/entities/non-existent-uuid');

```

```
expect(res.status).toBe(404);

});

});
```

File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Line #s
All files	94.44	75	100	94.11	
src	100	100	100	100	
app.ts	100	100	100	100	
... dleware	95.23	25	100	93.75	19
... ler.ts	85.71	25	100	83.33	
index.ts	100	100	100	100	
... ion.ts	100	100	100	100	
src/routes	89.18	100	100	89.18	
entity.ts	89.18	100	100	89.18	21,30,43,56
... schemas	100	100	100	100	
... ema.ts	100	100	100	100	
... storage	100	100	100	100	
entity.ts	100	100	100	100	
Test Suites: 1 passed, 1 total					
Tests: 14 passed, 14 total					
Snapshots: 0 total					
Time: 4.076 s, estimated 6 s					

3.7. Завдання 7: Фільтрація через query-параметри

src/routes/entity.ts

```
import { Router } from 'express';
import { entityStorage } from '../storage/entity';
import { createSchema, updateSchema, EntityFilters } from '../schemas/entity.schema';
import { validate } from '../middleware';

const router = Router();

router.get('/', (req, res) => {
  const { priority, maxPrice, name } = req.query;
  const filters: EntityFilters = {};
  if (typeof priority === 'string' && ['low', 'medium', 'high'].includes(priority)) {
    filters.priority = priority as 'low' | 'medium' | 'high';
  }
}
```

```

if (typeof maxPrice === 'string' && !isNaN(Number(maxPrice))) {
  filters.maxPrice = Number(maxPrice);
}
if (typeof name === 'string') {
  filters.name = name;
}
res.status(200).json(entityStorage.findAll(filters));
});

router.get('/:id', (req, res, next) => {
  try {
    const item = entityStorage.findById(req.params.id as string);
    if (!item) {
      res.status(404).json({ error: 'Item not found' });
      return;
    }
    res.status(200).json(item);
  } catch (error) {
    next(error);
  }
});

router.post('/', validate(createSchema), (req, res, next) => {
  try {
    const newItem = entityStorage.create(req.body);
    res.status(201).json(newItem);
  } catch (error) {
    next(error);
  }
});

router.put('/:id', validate(updateSchema), (req, res, next) => {
  try {

```

```

const updated = entityStorage.update(req.params.id as string, req.body);
if (!updated) {
  res.status(404).json({ error: 'Item not found' });
  return;
}
res.status(200).json(updated);
} catch (error) {
  next(error);
}
});

router.delete('/:id', (req, res, next) => {
  try {
    const success = entityStorage.delete(req.params.id as string);
    if (!success) {
      res.status(404).json({ error: 'Item not found' });
      return;
    }
    res.status(204).end();
  } catch (error) {
    next(error);
  }
});

export default router;

```

src/schemas/entity.schema.ts

```

import { z } from 'zod';

export const createSchema = z.object({
  name: z.string().min(1).max(100),
  description: z.string().max(500).optional(),

```

		Ігнатюк А.М.			ДУ «Житомирська політехніка».26.121.8.000 – ЛрЗ	Арк.
		Фвріхата Д.В.				15
Змн.	Арк.	№ докум.	Підпис	Дата		

```

    price: z.number().positive(),
    priority: z.enum(['low', 'medium', 'high']),
    url: z.string().url().optional()
  });

export const updateSchema = createSchema.partial();

export type Entity = z.infer<typeof createSchema> & {
  id: string;
  createdAt: Date;
  updatedAt: Date;
};

export interface EntityFilters {
  priority?: 'low' | 'medium' | 'high';
  maxPrice?: number;
  name?: string;
}

```

src/storage/entity.ts

```

import { randomUUID } from 'crypto';
import { Entity, EntityFilters } from '../schemas/entity.schema';

const storage = new Map<string, Entity>();

export const entityStorage = {
  findAll(filters?: EntityFilters): Entity[] {
    let results = Array.from(storage.values());

    if (!filters) {
      return results;
    }

    if (filters.priority) {

```

```

    results = results.filter(item => item.priority === filters.priority);
  }
  if (filters.maxPrice !== undefined) {
    results = results.filter(item => item.price <= filters.maxPrice!);
  }
  if (filters.name) {
    const search = filters.name.toLowerCase();
    results = results.filter(item => item.name.toLowerCase().includes(search));
  }
  return results;
},

findById(id: string): Entity | undefined {
  return storage.get(id);
},

create(item: Omit<Entity, 'id' | 'createdAt' | 'updatedAt'>): Entity {
  const id = randomUUID();
  const now = new Date();
  const newItem: Entity = {
    ...item,
    id,
    createdAt: now,
    updatedAt: now
  };
  storage.set(id, newItem);
  return newItem;
},

update(id: string, item: Partial<Omit<Entity, 'id' | 'createdAt' | 'updatedAt'>>): Entity |
undefined {
  const existing = storage.get(id);
  if (!existing) {
    return undefined;
  }

```

		Ігнатюк А.М.			ДУ «Житомирська політехніка».26.121.8.000 – Лр3	Арк.
		Фуріхата Л.В.				17
Змн.	Арк.	№ докум.	Підпис	Дата		

```

    }

    const updatedItem: Entity = {
      ...existing,
      ...item,
      updatedAt: new Date()
    };

    storage.set(id, updatedItem);

    return updatedItem;
  },

  delete(id: string): boolean {
    return storage.delete(id);
  },

  reset(): void {
    storage.clear();
  }
};

```

tests/entity.test.ts

```

import request from 'supertest';
import app from '../src/app';
import { entityStorage } from '../src/storage/entity';

describe('Wishlist API Integration Tests', () => {
  beforeEach(() => {
    entityStorage.reset();
  });

  const validItem = {
    name: 'PlayStation 5',
    description: 'Next-gen gaming console',

```

		Ігнатюк А.М.			ДУ «Житомирська політехніка».26.121.8.000 – Лр3	Арк.
		Фуріхата Д.В.				18
Змн.	Арк.	№ докум.	Підпис	Дата		

```

    price: 499.99,
    priority: 'high' as const,
    url: 'https://sony.com/ps5'
  };

it('GET /entities should return an empty array initially', async () => {
  const res = await request(app).get('/entities');
  expect(res.status).toBe(200);
  expect(res.body).toEqual([]);
});

it('GET /entities/:id should return 404 for non-existent ID', async () => {
  const res = await request(app).get('/entities/non-existent-uuid');
  expect(res.status).toBe(404);
  expect(res.body).toEqual({ error: 'Item not found' });
});

it('POST /entities should create a wishlist item and return 201', async () => {
  const res = await request(app).post('/entities').send(validItem);
  expect(res.status).toBe(201);
  expect(res.body).toHaveProperty('id');
  expect(res.body).toHaveProperty('createdAt');
  expect(res.body).toHaveProperty('updatedAt');
  expect(res.body.name).toBe(validItem.name);
  expect(res.body.description).toBe(validItem.description);
  expect(res.body.price).toBe(validItem.price);
  expect(res.body.priority).toBe(validItem.priority);
  expect(res.body.url).toBe(validItem.url);
});

it('POST /entities should return 400 if name is empty', async () => {
  const res = await request(app).post('/entities').send({ ...validItem, name: '' });
  expect(res.status).toBe(400);
});

```

```

expect(res.body.status).toBe('error');
expect(res.body.message).toBe('Validation error');
});

it('POST /entities should return 400 if name exceeds 100 characters', async () => {
  const longName = 'a'.repeat(101);
  const res = await request(app).post('/entities').send({ ...validItem, name: longName });
  expect(res.status).toBe(400);
  expect(res.body.status).toBe('error');
});

it('POST /entities should return 400 if description exceeds 500 characters', async () => {
  const longDesc = 'a'.repeat(501);
  const res = await request(app).post('/entities').send({ ...validItem, description: longDesc });
  expect(res.status).toBe(400);
});

it('POST /entities should return 400 if price is negative', async () => {
  const res = await request(app).post('/entities').send({ ...validItem, price: -10 });
  expect(res.status).toBe(400);
});

it('POST /entities should return 400 if priority is invalid', async () => {
  const res = await request(app).post('/entities').send({ ...validItem, priority: 'critical' });
  expect(res.status).toBe(400);
});

it('POST /entities should return 400 if url is invalid URL format', async () => {
  const res = await request(app).post('/entities').send({ ...validItem, url: 'not-a-valid-url' });
  expect(res.status).toBe(400);
});

it('GET /entities/:id should return the item if it exists', async () => {

```

```

const created = entityStorage.create(validItem);
const res = await request(app).get(`/entities/${created.id}`);
expect(res.status).toBe(200);
expect(res.body.id).toBe(created.id);
expect(res.body.name).toBe(validItem.name);
});

it('PUT /entities/:id should update wishlist item and return 200', async () => {
  const created = entityStorage.create(validItem);
  const updateData = { name: 'PlayStation 5 Slim', price: 449.99 };
  const res = await request(app).put(`/entities/${created.id}`).send(updateData);
  expect(res.status).toBe(200);
  expect(res.body.id).toBe(created.id);
  expect(res.body.name).toBe(updateData.name);
  expect(res.body.price).toBe(updateData.price);
  expect(res.body.priority).toBe(validItem.priority);
});

it('PUT /entities/:id should return 404 if updating non-existent item', async () => {
  const res = await request(app).put('/entities/non-existent-uuid').send({ name: 'New Name' });
  expect(res.status).toBe(404);
});

it('DELETE /entities/:id should delete item and return 204', async () => {
  const created = entityStorage.create(validItem);
  const delRes = await request(app).delete(`/entities/${created.id}`);
  expect(delRes.status).toBe(204);

  const getRes = await request(app).get(`/entities/${created.id}`);
  expect(getRes.status).toBe(404);
});

it('DELETE /entities/:id should return 404 if deleting non-existent item', async () => {

```

```

const res = await request(app).delete('/entities/non-existent-uuid');
expect(res.status).toBe(404);
});

it('GET /entities?priority=high should return only high-priority items', async () => {
  entityStorage.create({ ...validItem, name: 'Item 1', priority: 'high' });
  entityStorage.create({ ...validItem, name: 'Item 2', priority: 'low' });
  entityStorage.create({ ...validItem, name: 'Item 3', priority: 'high' });

  const res = await request(app).get('/entities?priority=high');
  expect(res.status).toBe(200);
  expect(res.body.length).toBe(2);
  expect(res.body.every((item: any) => item.priority === 'high')).toBe(true);
});

it('GET /entities?maxPrice=100 should return only items with price <= 100', async () => {
  entityStorage.create({ ...validItem, name: 'Cheap 1', price: 50 });
  entityStorage.create({ ...validItem, name: 'Expensive', price: 150 });
  entityStorage.create({ ...validItem, name: 'Cheap 2', price: 100 });

  const res = await request(app).get('/entities?maxPrice=100');
  expect(res.status).toBe(200);
  expect(res.body.length).toBe(2);
  expect(res.body.every((item: any) => item.price <= 100)).toBe(true);
});

it('GET /entities?name=Play should return only items with matching names (case-insensitive)', async () => {
  entityStorage.create({ ...validItem, name: 'PlayStation 5' });
  entityStorage.create({ ...validItem, name: 'Xbox Series X' });
  entityStorage.create({ ...validItem, name: 'Nintendo Switch' });

  const res = await request(app).get('/entities?name=play');
  expect(res.status).toBe(200);

```

```

    expect(res.body.length).toBe(1);

    expect(res.body[0].name).toBe('PlayStation 5');

  });

it('GET /entities with multiple filters should return only matching items', async () => {
  entityStorage.create({ ...validItem, name: 'Low Cheap', priority: 'low', price: 50 });
  entityStorage.create({ ...validItem, name: 'Low Expensive', priority: 'low', price: 150 });
  entityStorage.create({ ...validItem, name: 'High Cheap', priority: 'high', price: 50 });

  const res = await request(app).get('/entities?priority=low&maxPrice=100');

  expect(res.status).toBe(200);

  expect(res.body.length).toBe(1);

  expect(res.body[0].name).toBe('Low Cheap');

});

});

```

Тестування

File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Line #s
All files	94.59	86.66	100	94.17	
src	100	100	100	100	
app.ts	100	100	100	100	
src/middleware	95.23	25	100	93.75	
errorHandler.ts	85.71	25	100	83.33	19
index.ts	100	100	100	100	
validation.ts	100	100	100	100	
src/routes	91.11	100	100	91.11	
entity.ts	91.11	100	100	91.11	32,41,54,67
src/schemas	100	100	100	100	
entity.schema.ts	100	100	100	100	
src/storage	96.77	90	100	96.42	
entity.ts	96.77	90	100	96.42	10
Test Suites: 1 passed, 1 total					
Tests: 18 passed, 18 total					
Snapshots: 0 total					
Time: 2.608 s, estimated 6 s					

Висновок: в ході виконання роботи набув практичних навичок розробки RESTful API на Express.js з TypeScript та написання інтеграційних тестів з використанням Supertest, зокрема: • ініціалізація Express-проєкту з TypeScript та налаштування необхідних залежностей; • визначення типів сутності через Zod-схеми та z.infer; • реалізація in-memory сховища з використанням Map; • побудова CRUD-маршрутів з правильними HTTP-статус-кодами; • розділення конфігурації застосунку (app.ts) та запуску сервера (server.ts); • написання інтеграційних тестів для всіх маршрутів з використанням Supertest.

		Ігнатюк А.М.			ДУ «Житомирська політехніка».26.121.8.000 – ЛрЗ	Арк.
		Фвріхата Д.В.				24
Змн.	Арк.	№ докум.	Підпис	Дата		